

Solidity Contract Audit Report

Project: PasswordStore

Contract(s) Audited: `src/PasswordStore.sol`

Audit Date: June 21, 2026

Severity Scale: Critical > High > Medium > Low > Informational

Executive Summary

Three vulnerabilities were identified during the audit of the PasswordStore contract: - **2 High severity** issues (on-chain password storage, missing access control) - **1 Informational** issue (incorrect natspec)

The two High severity issues severely break the contract's intended functionality and should be addressed before deployment.

High Severity Findings

[H-1] Storing the password on-chain makes it visible to anyone and no longer private

Severity: HIGH

Impact: HIGH

Likelihood: HIGH

Description All data stored on-chain is visible to anyone and can be read directly from the blockchain. The `PasswordStore::s_password` variable is intended to be a private variable and only accessed through the `PasswordStore::getPassword` function, which is intended to be only called by the owner of the contract.

Impact Anyone can read the private password, severely breaking the functionality of the protocol.

Proof of Concept The following test case shows how anyone can read the password directly from the blockchain:

1. Deploy on Anvil:

```
anvil
```

```
make deploy
```

4. Retrieve password using Cast:

5. Parse to get myPassword:

```
cast parse-bytes32-string 0x6d7950617373776f726440000000000000000000000000000000
```

Recommended Mitigation Due to this, the overall architecture of the contract should be rethought. One approach: - Encrypt the password off-chain before storing, then decrypt when needed - Remove the view function to prevent users from accidentally sending transactions that expose the password - Consider using a different design pattern (e.g., commitment schemes, zero-knowledge proofs) if password privacy is required

Severity: HIGH
Impact: HIGH
Likelihood: HIGH

```
function setPassword(string memory newPassword) external {
    //@audit - There are no access controls
    s_password = newPassword;
    emit SetNewPassword();
}
```

2

Proof of Concept Add the following test to PasswordStore.t.sol:

```
function test_non_owner_can_set_password(address randomAddress) public {
    vm.assume(owner != randomAddress);
    vm.prank(randomAddress);
    string memory expectedPassword = "myPassword";
    passwordStore.setPassword(expectedPassword);

    vm.prank(owner);
    string memory actualPassword = passwordStore.getPassword();
    assertEq(actualPassword, expectedPassword);
}
```

This test will pass, demonstrating that a non-owner can change the password.

Recommended Mitigation Add an access control conditional to the setPassword function:

```
function setPassword(string memory newPassword) external {
    if(msg.sender != s_owner) {
        revert PasswordStore__NotOwner();
    }
    s_password = newPassword;
    emit SetNewPassword();
}
```

Alternatively, use OpenZeppelin's Ownable modifier:

```
function setPassword(string memory newPassword) external onlyOwner {
    s_password = newPassword;
    emit SetNewPassword();
}
```

Informational Findings

[S-1] The PasswordStore::getPassword NatSpec indicates a parameter that doesn't exist

Severity: INFORMATIONAL

Impact: NONE

Likelihood: HIGH

```
/*
 * @notice This allows only the owner to retrieve the password.
 * // @audit — there is no newPassword parameter!
 * @param newPassword The new password to set.
```

```

    */
function getPassword() external view returns (string memory) {

```

The PasswordStore::getPassword function is getPassword() (no parameters), but the NatSpec says it should be getPassword(string) with a newPassword parameter.

Impact The NatSpec is incorrect and may confuse developers.

Recommended Mitigation Remove the incorrect NatSpec line:

```

- * @param newPassword The new password to set.

```

Updated NatSpec:

```

/*
    * @notice This allows only the owner to retrieve the password.
    */
function getPassword() external view returns (string memory) {

```

Note: This may indicate a design issue. Review whether the function should be renamed or if the NatSpec was copied from setPassword.

Summary Table

ID	Finding	Severity	Impact	Likelihood
H-1	On-chain password storage is visible to anyone	HIGH	HIGH	HIGH
H-2	setPassword lacks access control	HIGH	HIGH	HIGH
S-1	Incorrect NatSpec parameter	INFORMATIONAL	NONE	HIGH

Recommendations

1. **Address H-1 immediately:** On-chain storage cannot be private. Consider off-chain encryption or a different architecture.
2. **Address H-2 before deployment:** Add onlyOwner or manual access control check to setPassword.
3. **Fix S-1 for code quality:** Update NatSpec to match the actual function signature.

Audit conducted using Foundry testing framework and manual code review.